Principles of Distributed Database Systems

M. Tamer Özsu
Patrick Valduriez

Outline

- Introduction
- Distributed and Parallel Database Design
- Distributed Data Control
- Distributed Query Processing
- Distributed Transaction Processing
- Data Replication
- Database Integration – Multidatabase Systems
- Parallel Database Systems
- Peer-to-Peer Data Management
- Big Data Processing
- NoSQL, NewSQL and Polystores
- Web Data Management

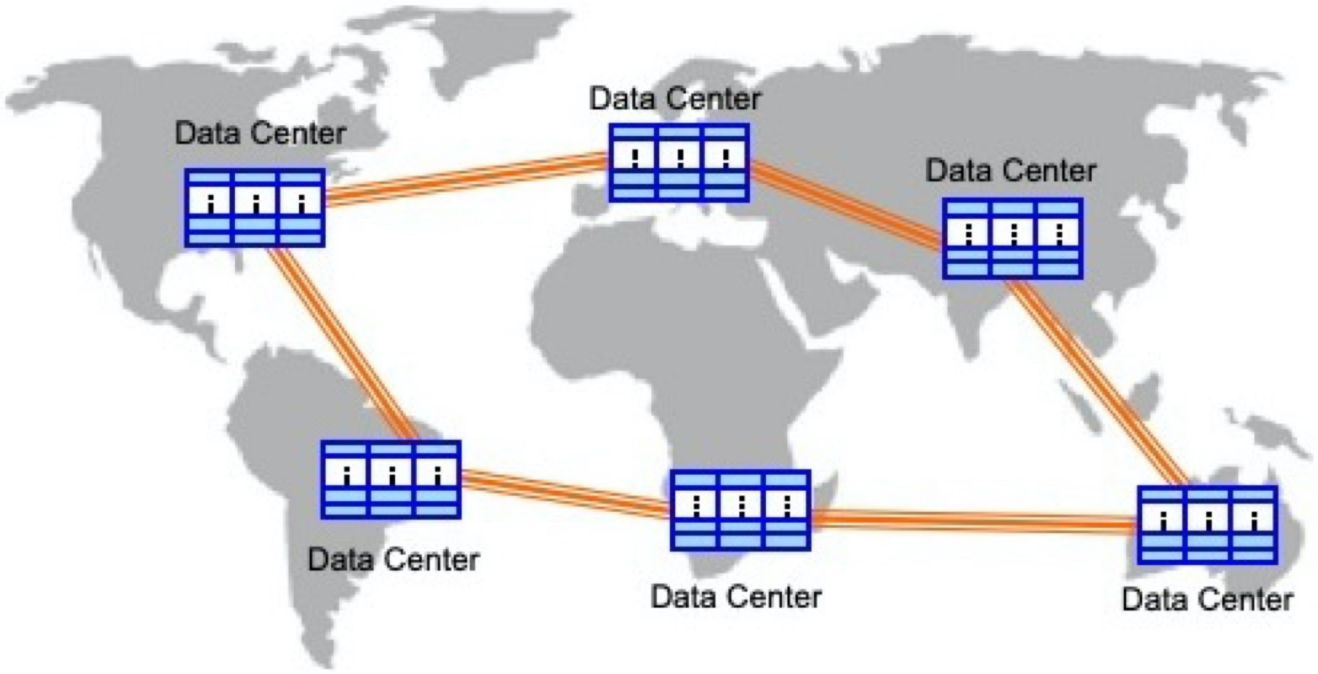
Outline

- Introduction
  - What is a distributed DBMS
  - History
  - Distributed DBMS promises
  - Design issues
  - Distributed DBMS architecture

Distributed Computing

- A number of autonomous processing elements (not necessarily homogeneous) that are interconnected by a computer network and that cooperate in performing their assigned tasks.
- What is being distributed?
  - Processing logic
  - Function
  - Data
  - Control

Current Distribution – Geographically Distributed Data Centers



What is a Distributed Database System?

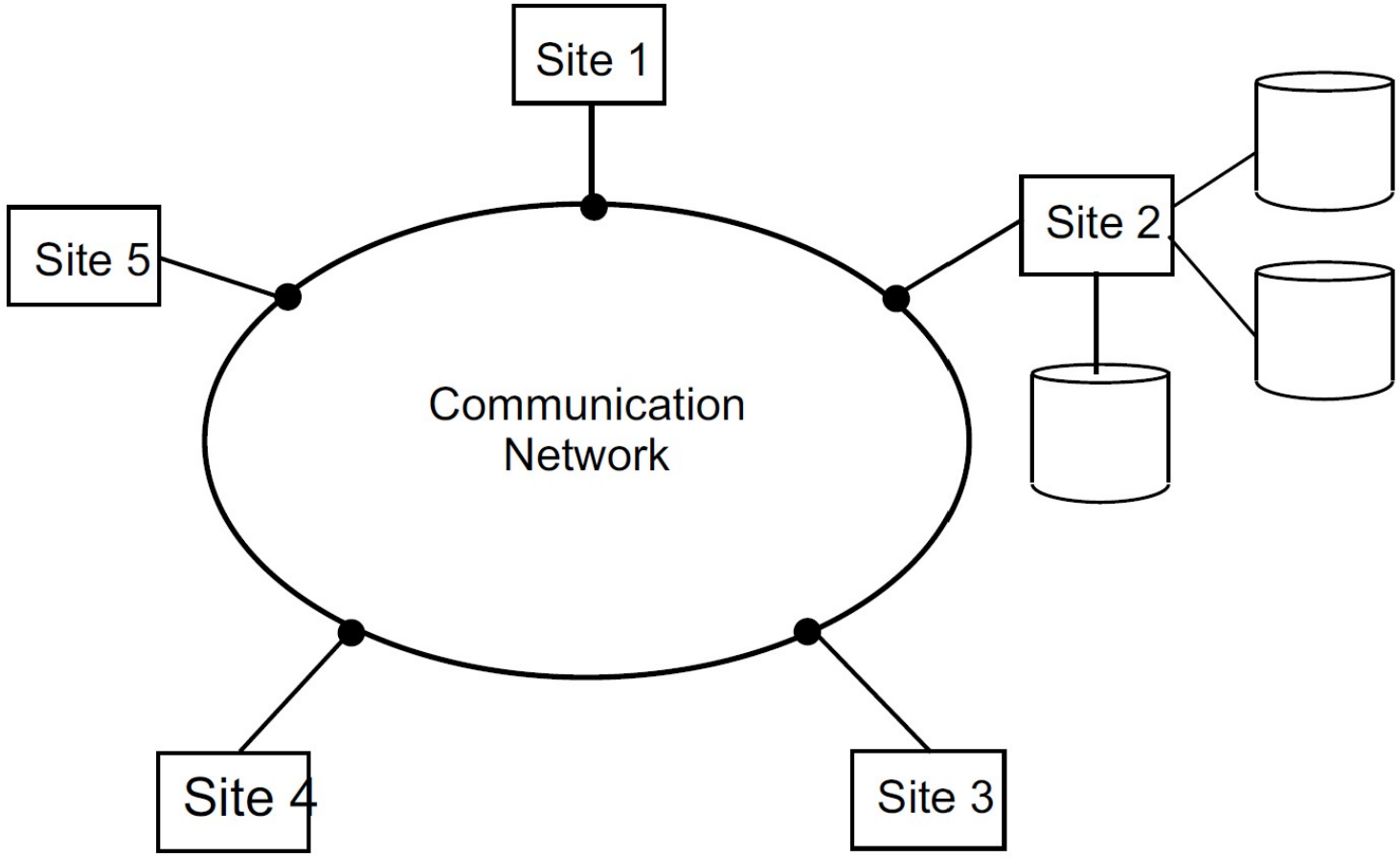
A distributed database is a collection of multiple, logically interrelated databases distributed over a computer network

A distributed database management system (Distributed DBMS) is the software that manages the DDB and provides an access mechanism that makes this distribution transparent to the users

What is not a DDBS?

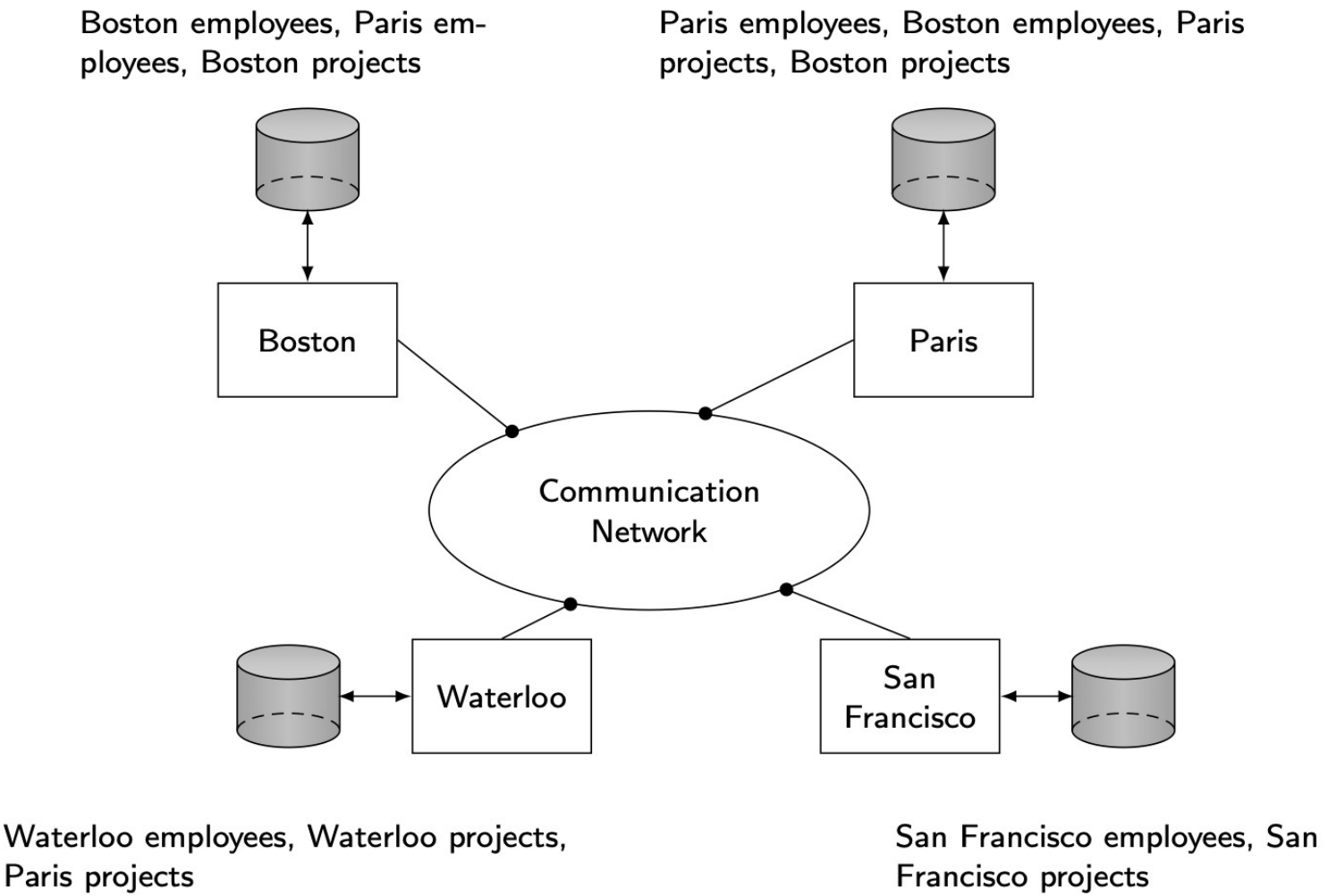
- A timesharing computer system
- A loosely or tightly coupled multiprocessor system
- A database system which resides at one of the nodes of a network of computers - this is a centralized database on a network node

Centralized Database on a Network



Central Database on a Network

Distributed DBMS Environment



Implicit Assumptions

- Data stored at a number of sites → each site *logically* consists of a single processor
- Processors at different sites are interconnected by a computer network → not a multiprocessor system
  - Parallel database systems
- Distributed database is a database, not a collection of files → data logically related as exhibited in the users' access patterns
  - Relational data model
- Distributed DBMS is a full-fledged DBMS
  - Not remote file system, not a TP system

Important Point

Logically integrated

but

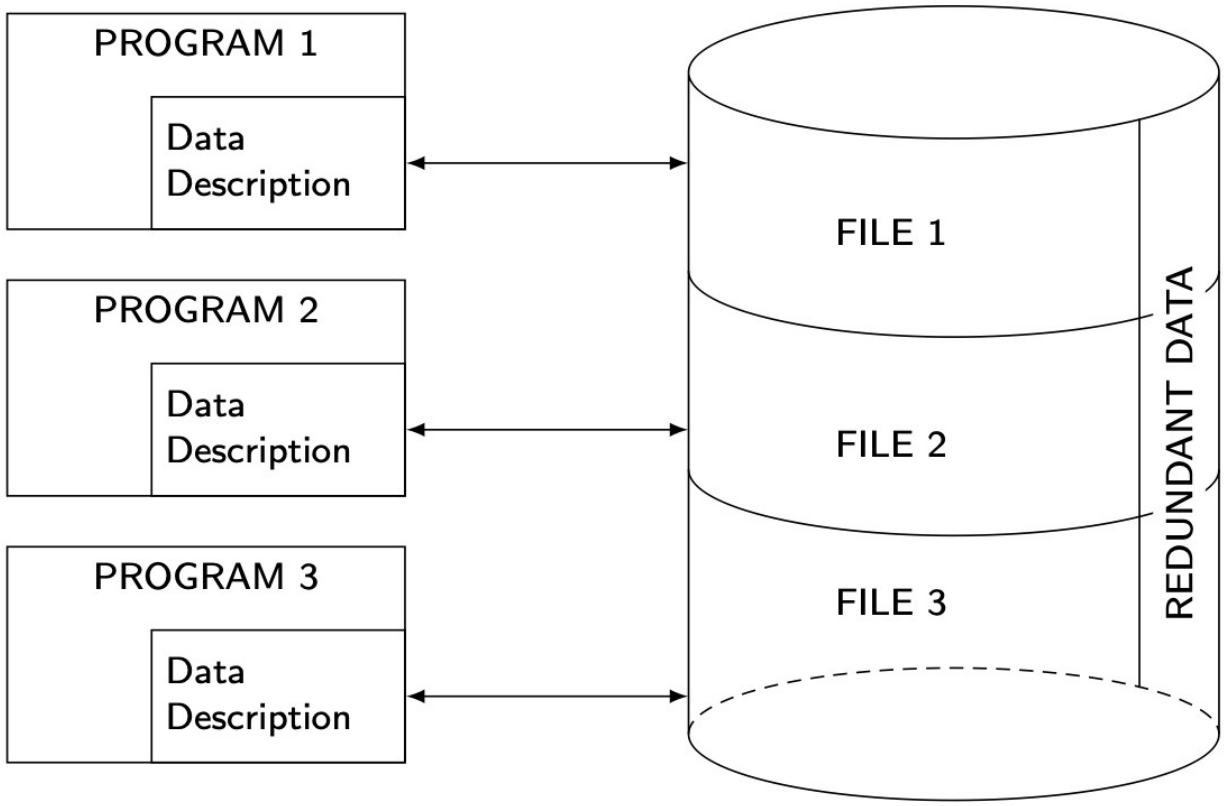
Physically distributed

Outline

- Introduction
  - What is a distributed DBMS
  - History
  - Distributed DBMS promises
  - Design issues
  - Distributed DBMS architecture

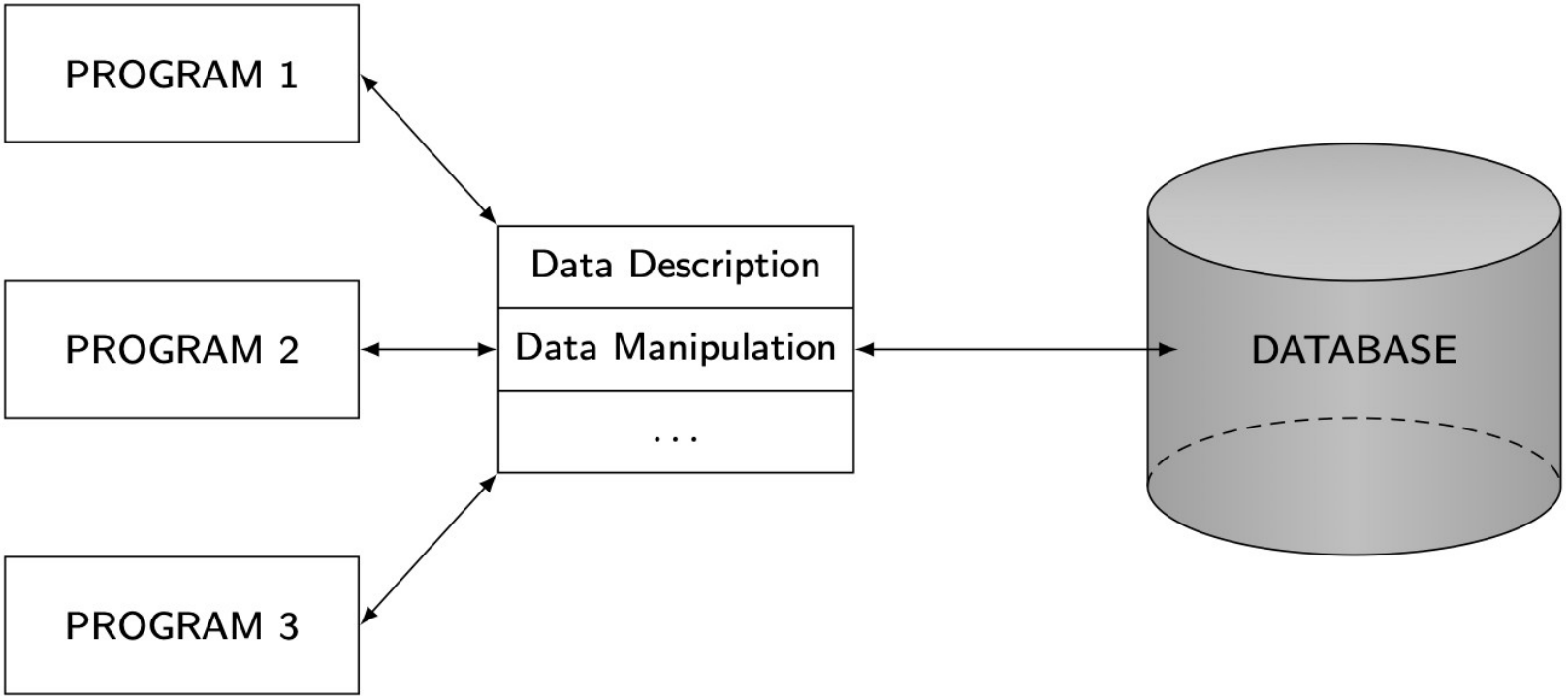
History – File Systems

Data stored in flat files; no query language, no distribution support (1960s)



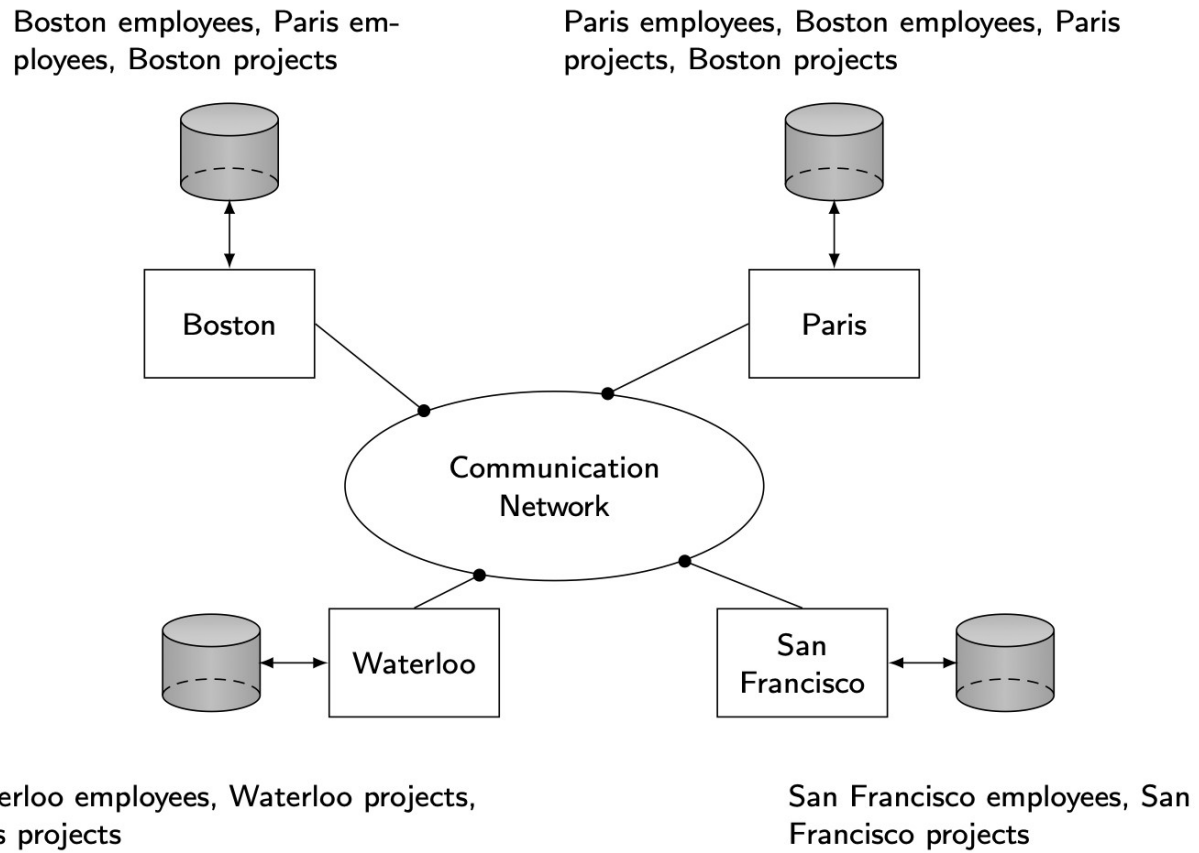
History – Database Management

Relational DBMS emerged (1970s)



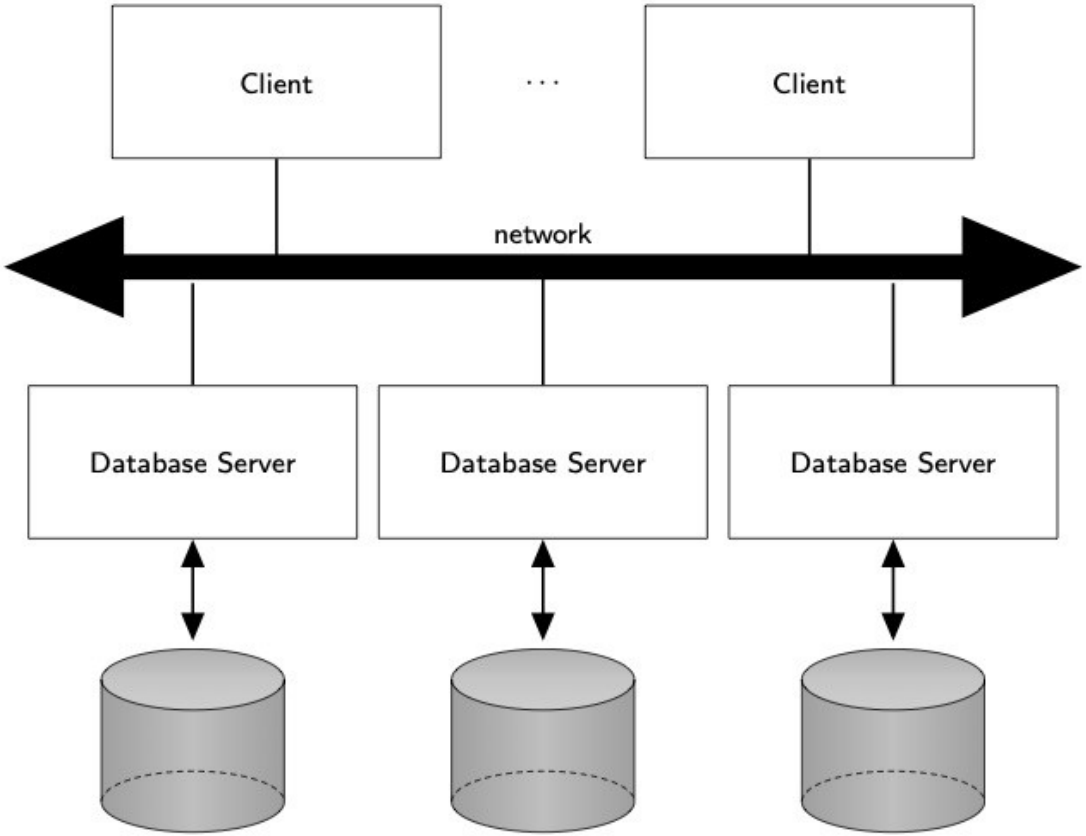
History – Early Distribution

Peer-to-Peer (P2P) **(1980s):** Nodes communicated directly; inspired distributed data sharing but lacked global control.



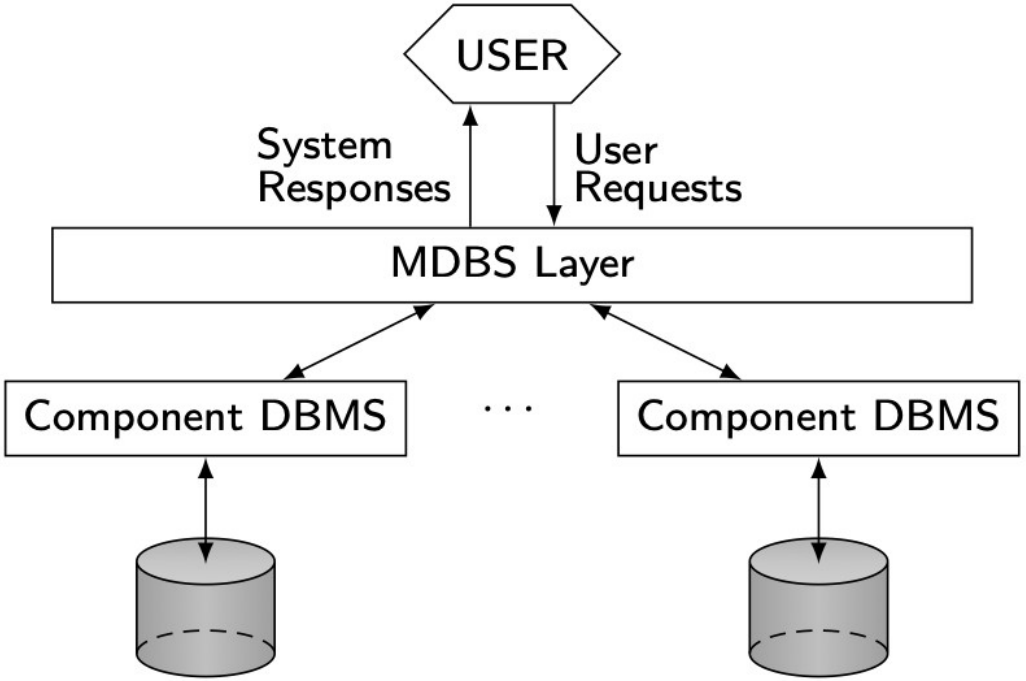
History – Client/Server

Databases split between client interface and server storage; enabled distributed queries (1990s)



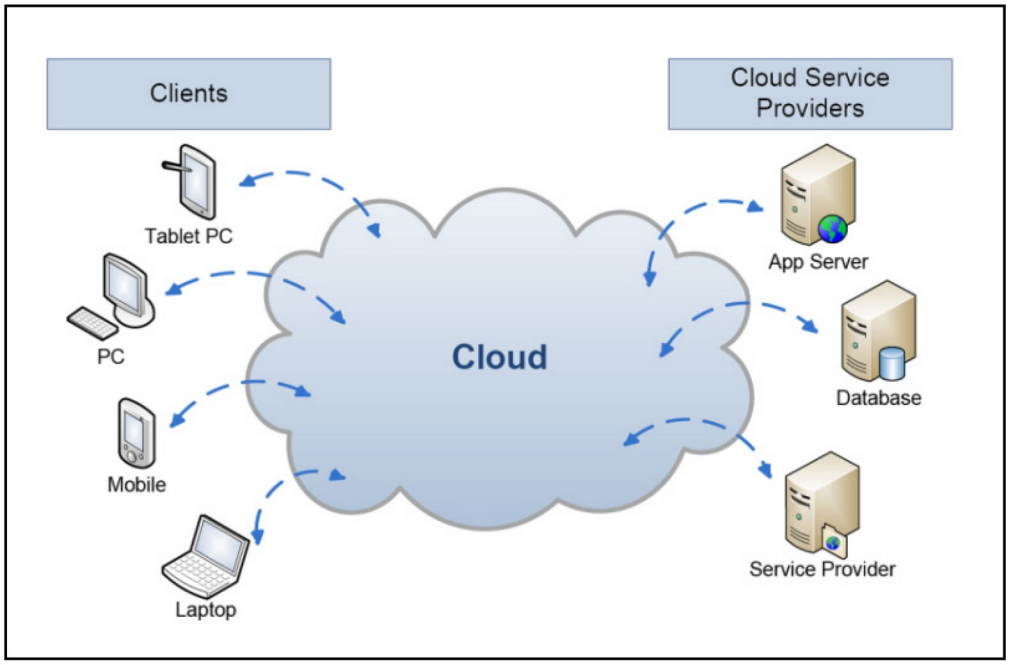
History – Data Integration

Middleware allowed multiple heterogeneous DBs to work as one system (2000s)



History – Cloud Computing

Cloud-native distributed databases provide global scale, elasticity, and high availability (2010s)



Data Delivery Alternatives

- Delivery modes
  - Pull-only
  - Push-only
  - Hybrid
- Frequency
  - Periodic
  - Conditional
  - Ad-hoc or irregular (Pull-only)
- Communication Methods
  - Unicast
  - One-to-many (Broadcast or Multicast)
- Note: not all combinations make sense

Outline

- Introduction
  - What is a distributed DBMS
  - History
  - Distributed DBMS promises
  - Design issues
  - Distributed DBMS architecture

Distributed DBMS Promises/features

1. Transparent management of distributed, fragmented, and replicated data
2. Improved reliability/availability through distributed transactions
3. Improved performance
4. Easier and more economical system expansion

1. Transparency

- Transparency is the separation of the higher-level semantics of a system from the lower level implementation issues.
- Data independence is a fundamental form of transparency in the distributed environment.
- Data independence means user applications remain unaffected by changes in data definitions or organization.

Types of Data independence :

- Logical Data Independence → Immunity of applications from changes in schema/logical structure.
- Physical Data Independence → Immunity of applications from changes in physical storage/organization.

Example

EMP

ENO	ENAME	TITLE
E1	J. Doe	Elect. Eng
E2	M. Smith	Syst. Anal.
E3	A. Lee	Mech. Eng.
E4	J. Miller	Programmer
E5	B. Casey	Syst. Anal.
E6	L. Chu	Elect. Eng.
E7	R. Davis	Mech. Eng.
E8	J. Jones	Syst. Anal.

ASG

ENO	PNO	RESP	DUR
E1	P1	Manager	12
E2	P1	Analyst	24
E2	P2	Analyst	6
E3	P3	Consultant	10
E3	P4	Engineer	48
E4	P2	Programmer	18
E5	P2	Manager	24
E6	P4	Manager	48
E7	P3	Engineer	36
E8	P3	Manager	40

PROJ

PNO	PNAME	BUDGET
P1	Instrumentation	150000
P2	Database Develop.	135000
P3	CAD/CAM	250000
P4	Maintenance	310000

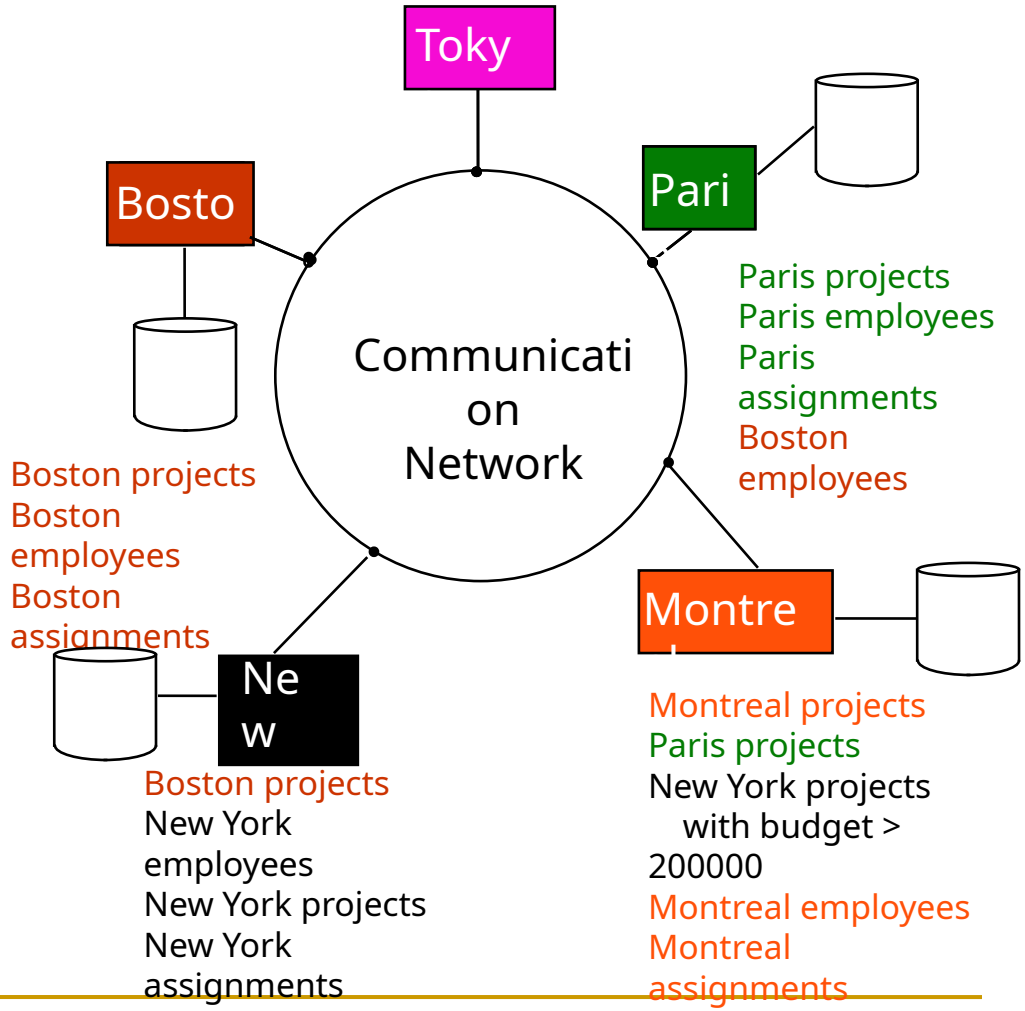
PAY

TITLE	SAL
Elect. Eng.	40000
Syst. Anal.	34000
Mech. Eng.	27000
Programmer	24000

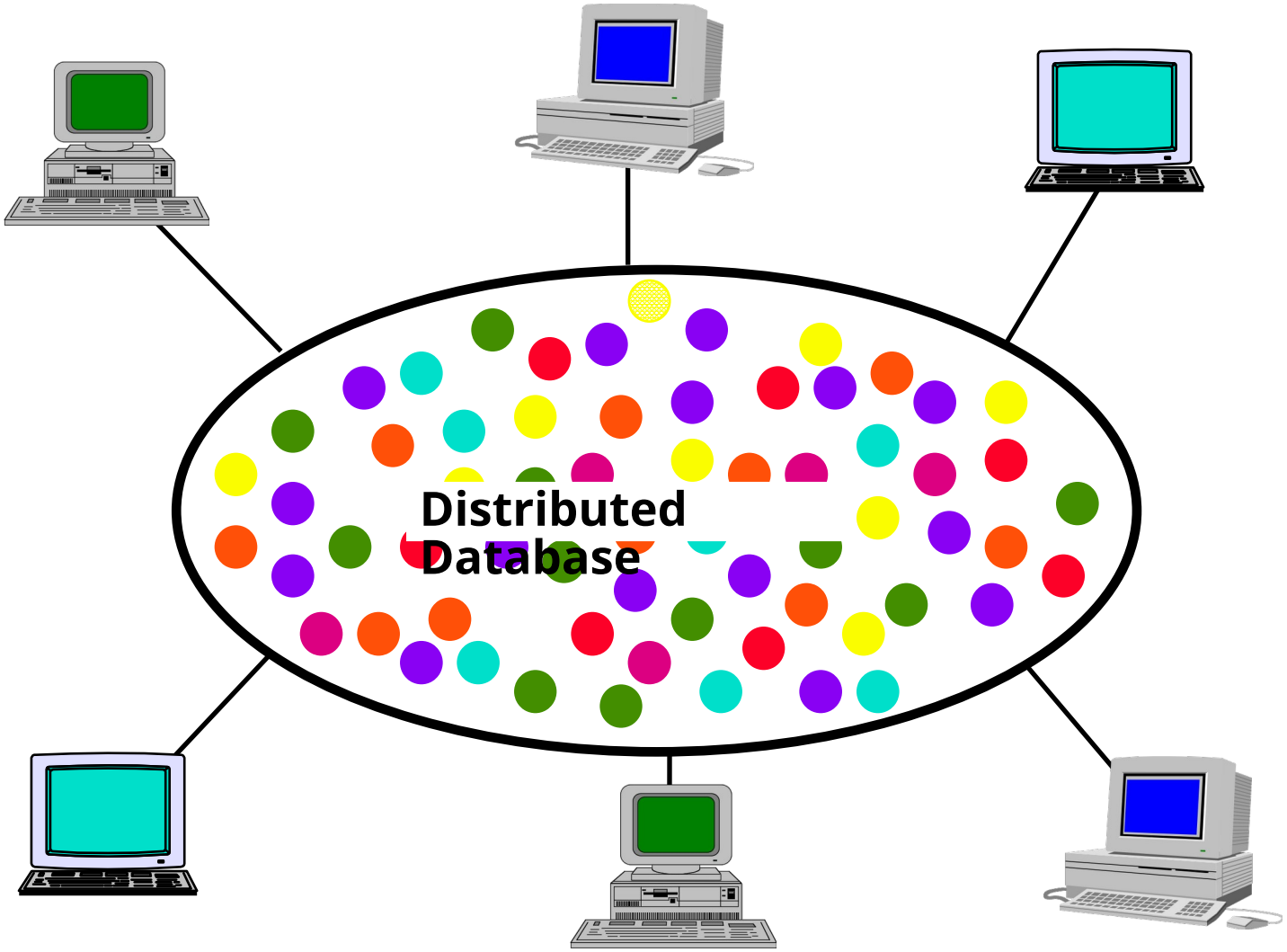
Transparent Access

SQL query to find out the names and salaries of employees who worked on a project for more than 12 months

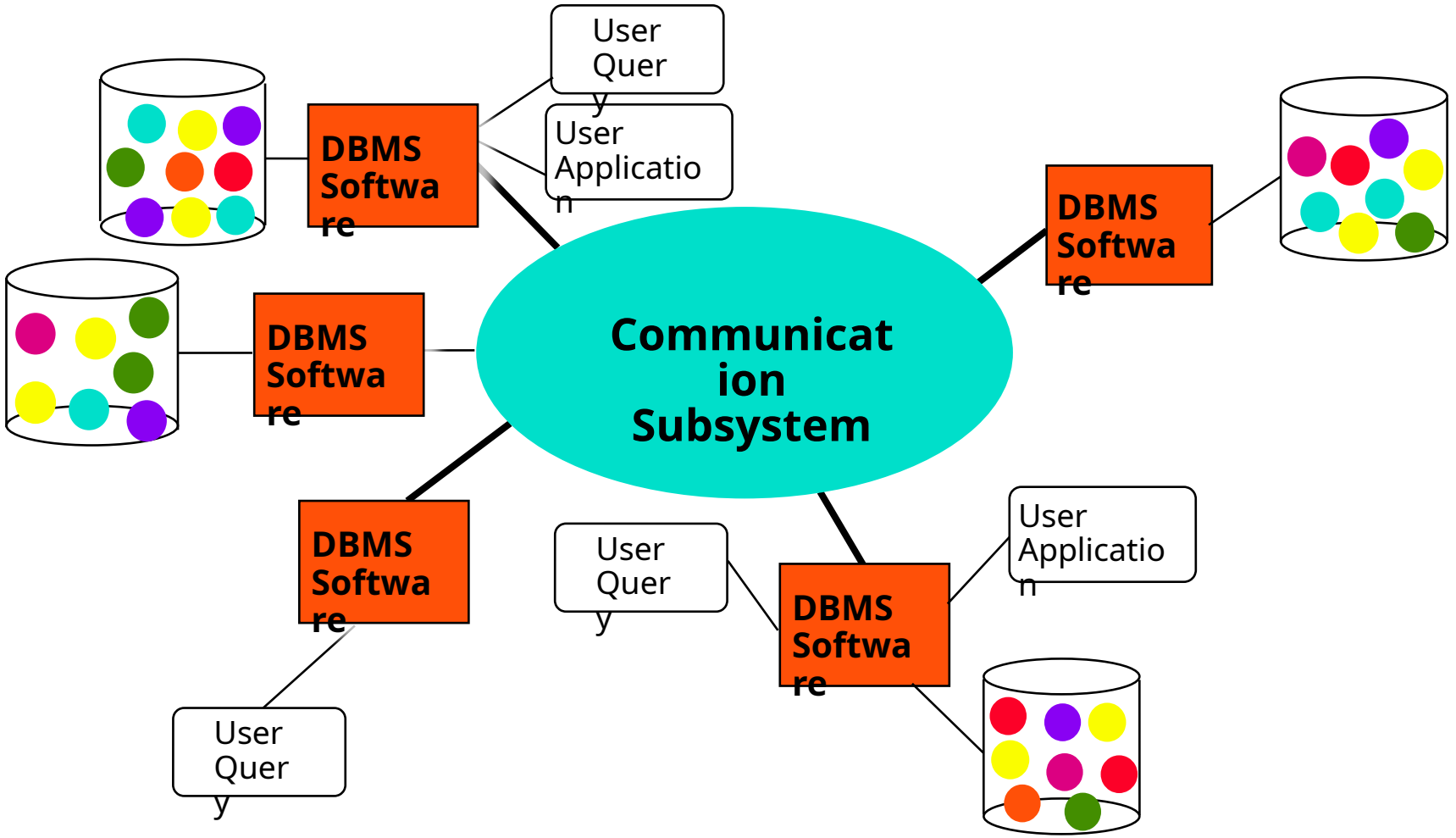
```
SELECT ENAME,SAL
FROM EMP,ASG,PAY
WHERE DUR > 12
AND EMP.ENO = ASG.ENO
AND PAY.TITLE = EMP.TITLE
```



Distributed Database - User View



Distributed DBMS - Reality



Types of Transparency

- Network transparency (or distribution transparency)
  - Location transparency
  - Fragmentation transparency
- Fragmentation transparency
  - horizontal fragmentation: selection
  - vertical fragmentation: projection
  - hybrid
- Replication transparency

Types of Transparency (Cont.)

Network Transparency

- Shields users from network details in distributed DBMS.
- Makes distributed DB appear like a centralized one.
- **Location Transparency** → Users need not know where data is stored.
- **Naming Transparency** → Unique name for each object; users need not include location.

Types of Transparency (Cont.)

Replication Transparency

- Data can be **replicated** across sites for performance, reliability, and availability.
- User should not be aware of multiple copies.
- System manages replication automatically.
- Ensures continuous access even if one site fails.

Types of Transparency (Cont.)

Fragmentation Transparency

- A relation may be **divided into fragments** (smaller relations).
- **Horizontal Fragmentation** → Divide by rows (tuples).
- **Vertical Fragmentation** → Divide by columns (attributes).
- Queries on full relation are automatically **translated to fragment queries**.

Types of Transparency (Cont.)

Who Provides Transparency?

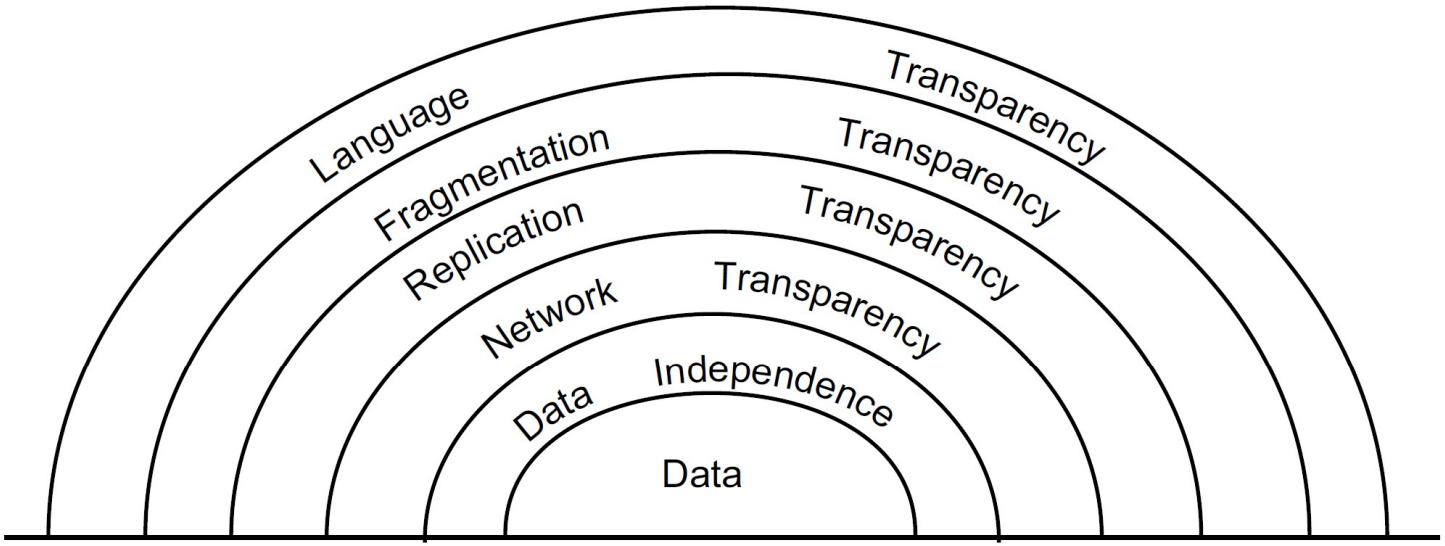
- **Access Layer** → User language/compiler translates requests.
- **Operating System Layer** → Distributed OS or middleware hides details.
- **DBMS Layer** → Most common; DBMS handles transparency for users.

- **Tradeoff** → More transparency = easier for users, but harder for DBMS.

Types of Transparency (Cont.)

Transparency Hierarchy

- Data Transparency
- Network Transparency
- Replication Transparency
- Fragmentation Transparency
- (Optional) Language Transparency (user-friendly query languages)



2.Reliability Through Transactions

Reliability in Distributed DBMS

- Eliminates single points of failure with replication.
- If one site/link fails → system continues with remaining sites.
- Ensures higher availability & fault tolerance.

Distributed Transactions

- **Transaction** = sequence of DB operations executed atomically.
- Guarantees consistency even with concurrency & failures.
- Example: Salary update across multiple sites must succeed as one unit.

2.Reliability Through Transactions (Cont)

Key Properties

- **Failure Atomicity** → Recovers to consistent state after failure.
- **Concurrency Transparency** → Concurrent transactions don't violate consistency.
- **Begin–End Boundaries** ensure safe execution.

Protocols for Reliability

- Distributed Concurrency Control protocols
- **Two-Phase Commit (2PC)** for atomic commit.
- **Replica Control** for managing copies.
- **Distributed Recovery Protocols** ensure system restoration.

3.Potentially Improved Performance

Improved Performance in Distributed DBMS

- Distributed DBMS fragments data → closer to point of use (**data localization**).
- Benefits:
  - Reduced **CPU/I/O contention**.
  - Reduced **remote access delays**.

Data Localization

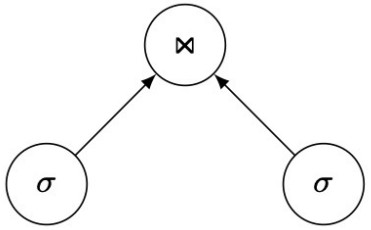
- Each site handles only part of DB → less load.
- Minimizes wide-area network delays (e.g., satellite: ~1 sec round-trip).
- Effective **fragmentation** + replication needed for max benefits.

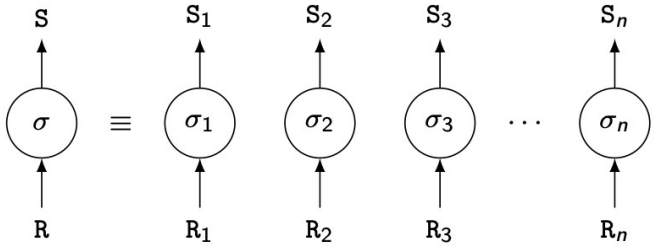
3.Potentially Improved Performance (Cont.)

- Parallelism in execution
  - Inter-query parallelism

    multiple queries run at the same time.
  - Intra-query parallelism

    single query split into subqueries at different sites.





4. Scalability

- Database size increase handled by adding processing & storage power.
- Scale-out: add more servers
  - Scale-up: increase the capacity of one server → has limits
- Benefits
  - Cost-Effective: Smaller computers + workstations cheaper than one big machine.
  - Economics: Grosh's Law (more power = disproportionately higher cost) no longer valid.
  - Practical: Distributed systems often more feasible than centralized mainframes.

Outline

- Introduction
  - What is a distributed DBMS
  - History
  - Distributed DBMS promises
  - Design issues
  - Distributed DBMS architecture

Distributed DBMS Issues

Distributed database design

- **Data Placement Approaches**
  - Partitioned (non-replicated): DB split into disjoint fragments at sites.
  - Replicated:
    - Fully → Entire DB at all sites.
    - Partially → Fragments stored at multiple (not all) sites.
- **Key Design Issues:**
  - **Fragmentation** → Breaking DB into fragments.
  - **Distribution** → Optimal placement of fragments.
- **Challenge:** NP-hard → heuristic-based solutions.

Distributed DBMS Issues

- Distributed query processing & Directory Management
  - Convert user transactions to data manipulation instructions
  - Optimization problem
    - min{cost = data transmission + local processing}
  - Consider: data distribution, comm. costs, parallelism.
- Directory Management:
  - Contains locations & descriptions of data.
  - Options: centralized, distributed, global/local, single/multiple copies.

Distributed DBMS Issues

- Distributed concurrency control
  - Synchronization of concurrent accesses
  - Maintain integrity across sites & replicas.
  - Approaches:
    - Pessimistic (locking, sync before exec).
    - Optimistic (check consistency after exec).
- Deadlock management
  - Similar to OS deadlocks.
  - Solutions: prevention, avoidance, detection & recovery.
- Reliability
  - Ensure DB consistency during site/network failures.
  - Recovery protocols handle failures & partitions.

Distributed DBMS Issues

- Replication
  - Mutual consistency
  - Freshness of copies
  - Eager vs lazy
    - Eager: update all replicas before commit.
    - Lazy: update master first, then propagate.
  - Centralized vs distributed

Related Issues

Interdependencies:

- Design ↔ Directory ↔ Query Processing.
- Replication ↔ Concurrency ↔ Reliability.

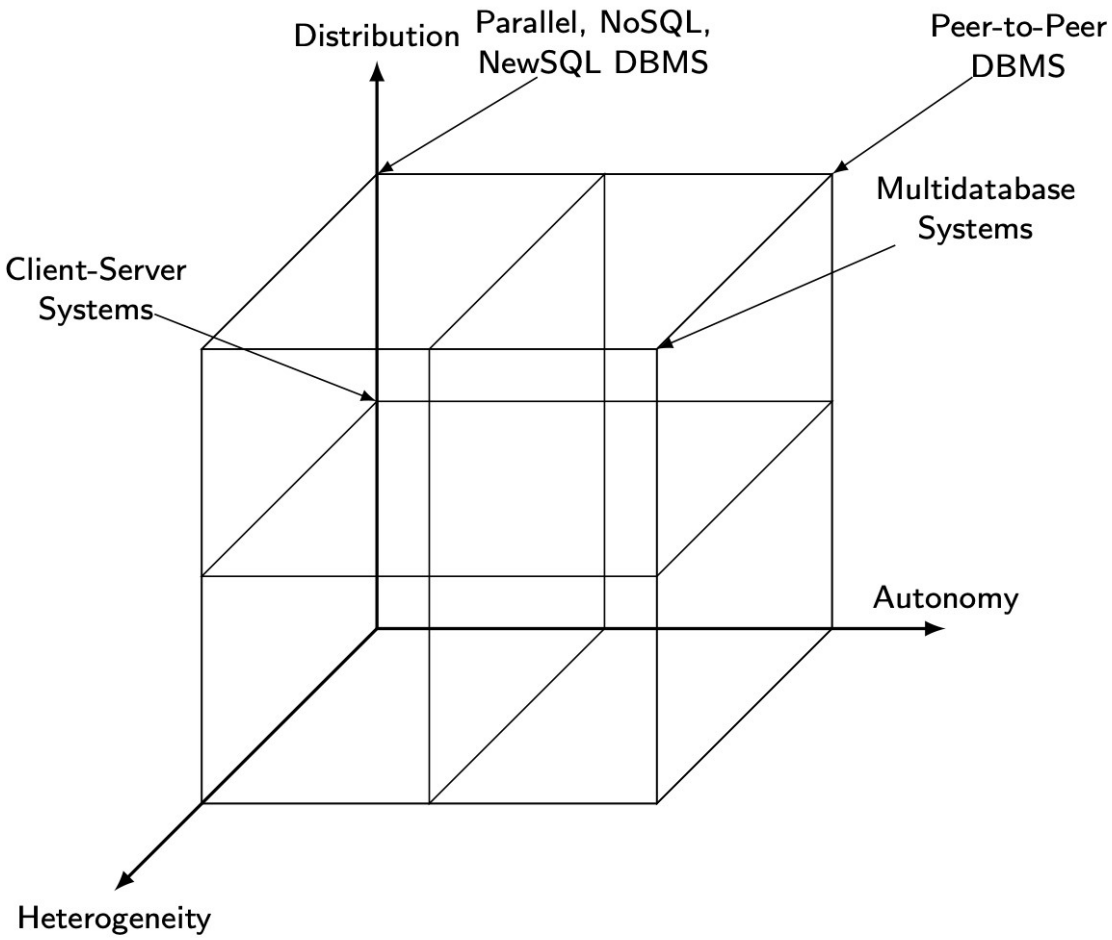
New Challenges:

- Federated / Multidatabase systems (heterogeneous integration).
- Peer-to-Peer (P2P) data management.
- Web Data Management (WWW & Internet growth).
- Parallel Databases within single logical sites.

Outline

- Introduction
  - What is a distributed DBMS
  - History
  - Distributed DBMS promises
  - Design issues
  - Distributed DBMS architecture

DBMS Implementation Alternatives



Dimensions of the Problem

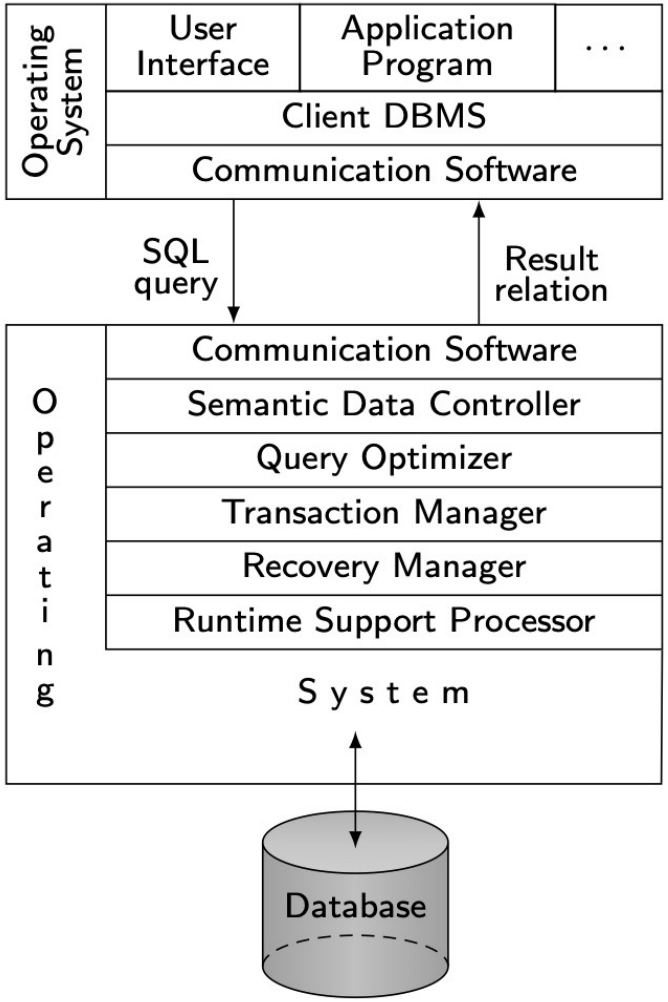
- Distribution (data)
  - Whether the components of the system are located on the same machine or not
    - Client/Server → Servers manage data; clients manage UI and apps.
    - Peer-to-Peer (P2P) → Each site is a full DBMS, equal participants.
    - Centralized → Not distributed.
- Heterogeneity (differences)
  - Various levels (hardware, communications, operating system)
  - Differences in **data models, query languages, transaction protocols**.
- Autonomy (control)
  - Not well understood and most troublesome
  - Various versions
    - Design autonomy: Ability of a component DBMS to decide on issues related to its own design.
    - Communication autonomy: Ability of a component DBMS to decide whether and how to communicate with other DBMSs.
    - Execution autonomy: Ability of a component DBMS to execute local operations in any manner it wants to.

DBMS Implementation Alternatives

Architectural Alternatives (3 main types)

1. Client/Server DBMS
2. Peer-to-Peer DBMS
3. Multidatabase Systems (MDBS)

Client/Server Architecture



Client/Server Architecture

The system is divided into two main parts:

1. Client Side (Front-End)

User Interface / Application Program

What the end-user sees and interacts with (e.g., forms, reports, GUIs).

Client DBMS

A lightweight DBMS component installed on the client machine.

Handles query formulation and presentation of results.

Communication Software (Client-side)

Sends SQL queries to the server and receives results.

- Manages the client-server connection.

Client/Server Architecture

2. Server Side (Back-End)-This is where the **heavy DBMS processing** happens.

Communication Software (Server-side)

Receives SQL queries from clients.

Sends back result sets after processing.

Semantic Data Controller

Ensures query meaning (semantics) is correct.

Checks schema, integrity constraints, and access rights.

Query Optimizer

Chooses the most efficient way to execute a query (e.g., which index to use, join method, etc.).

Transaction Manager

Ensures **ACID properties** of transactions (Atomicity, Consistency, Isolation, Durability).

Handles concurrency control (multiple users updating data at the same time).

Recovery Manager

Ensures database can recover from crashes/failures.

Uses logs and checkpoints.

Runtime Support Processor

Executes low-level DB operations (retrieving rows, updating tables, etc.).

Advantages of Client-Server Architectures

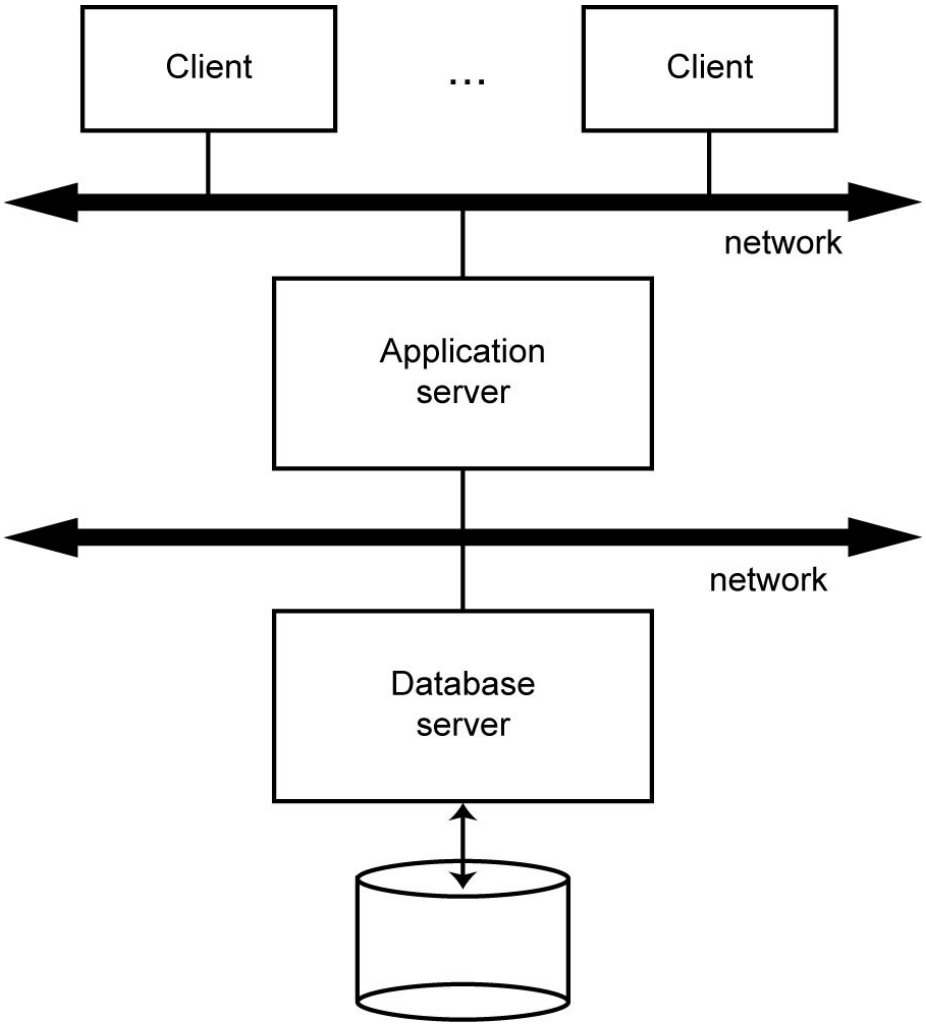
- More efficient division of labor
- Client access to remote data
- Horizontal and vertical scaling of resources
- Ability to use familiar tools on client machines
- Full DBMS functionality provided to client workstations
- Overall better system price/performance

Drawback: Extra communication overhead between client & server

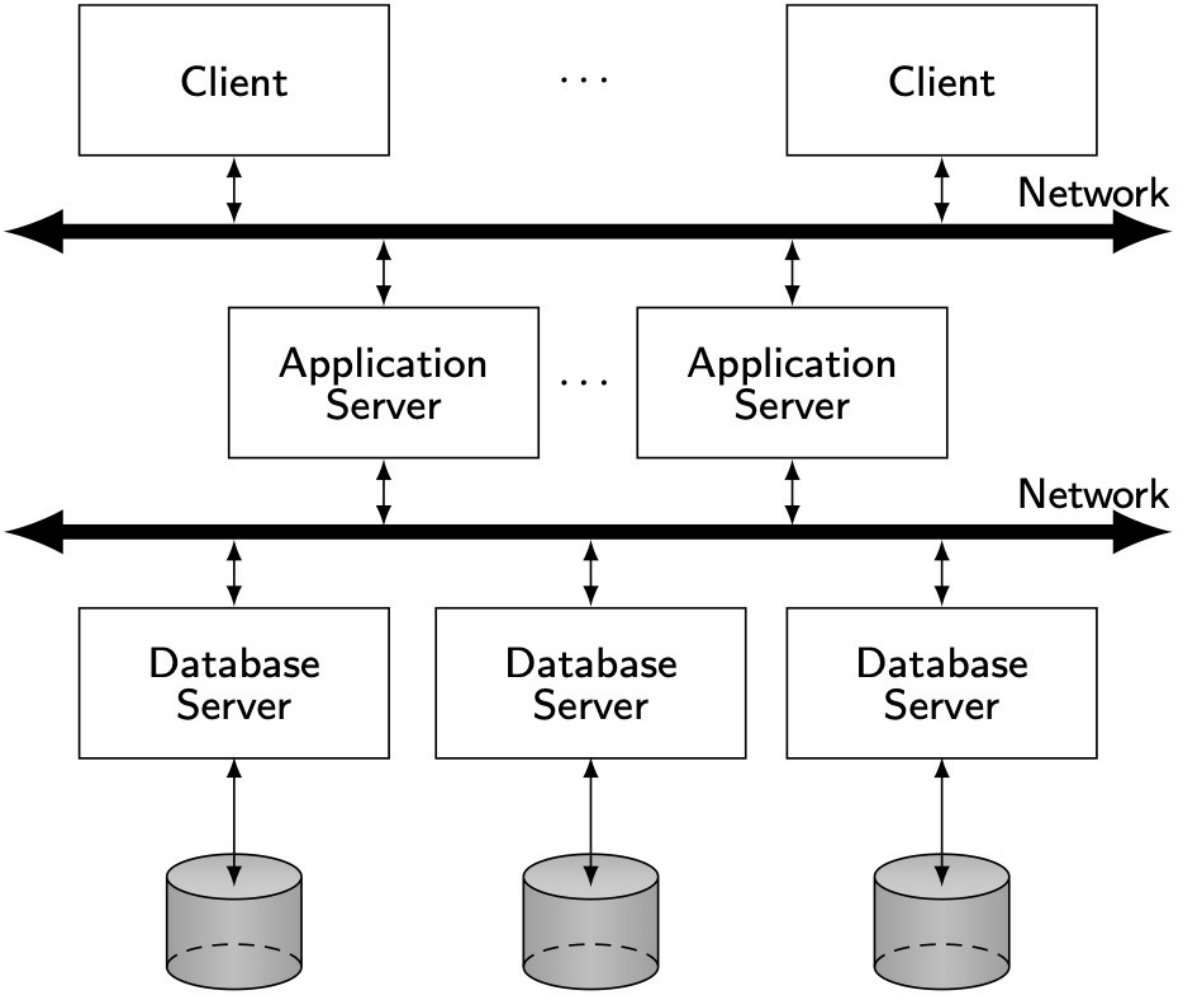
Extensions

- Multiple servers for scalability
- 3-tier architecture → UI, application servers, DB servers

3-tier structure - Database Server

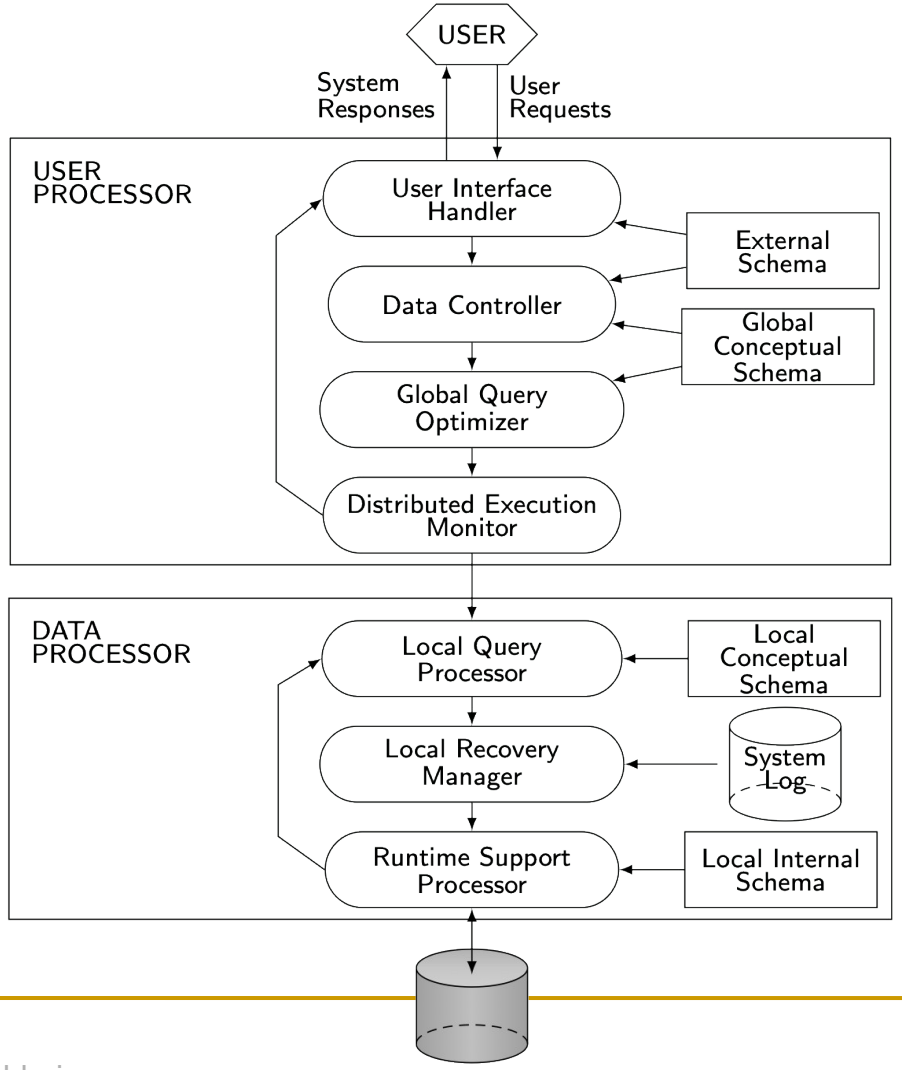


3-tier structure - Distributed Database Servers



Peer-to-Peer Component Architecture

Each site = complete DBMS (handles both users + data)



Peer-to-Peer Component Architecture

User Processor (Front-End)

Role: Interfaces with the user, handles query formulation, and coordinates distributed query execution.

Components:

- **User Interface Handler:** Accepts user requests & delivers results.
- **Data Controller:** Manages metadata (external & global conceptual schemas).
  - External Schema (User Views) describes how individual users or applications see the data.
  - Global Conceptual Schema (Logical View of Entire DB) - describes the logical structure of the whole database (all entities, relationships, constraints) as understood by the organization.
- **Global Query Optimizer:** Creates efficient distributed query execution plans.
- **Distributed Execution Monitor:** Coordinates query execution across multiple sites.
- **Key Idea:** Converts **user request → optimized distributed query → execution plan**.

Peer-to-Peer Component Architecture

Data Processor (Back-End)

Role: Executes the distributed query on local databases.

Components:

Local Query Processor: Executes queries based on **local conceptual schema**.
Local Recovery Manager: Ensures reliability, logging, and recovery at each site.
Runtime Support Processor: Handles physical data access (local internal schema).
Database Storage: Actual data resides in local DBs, managed independently.

Key Idea: Executes and manages **local queries, recovery, and data access** while ensuring global consistency.

Peer-to-Peer Component Architecture

Pros & Cons

✅ No central bottleneck (every node is equal)

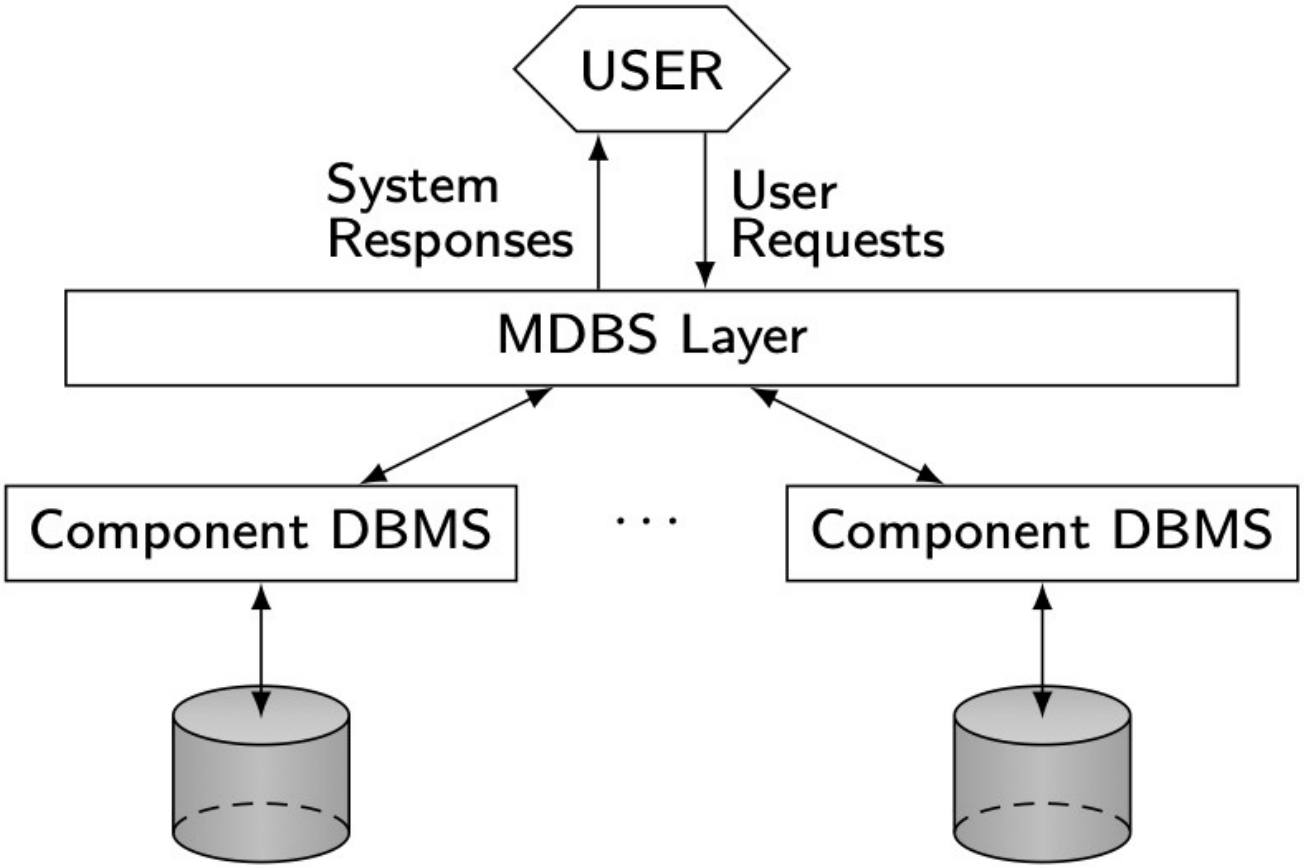
✅ High autonomy and transparency

❌ Complex coordination & consistency management

❌ Scalability challenges with heterogeneity

MDBS Architecture

Enables **transparent access** to multiple heterogeneous databases through a single interface



MDBS Components & Execution

1. USER

End users who need to access data
Submit queries and receive results
Don't need to know about multiple databases

2. MDBS Layer (Multidatabase Management System)

Core component that acts as middleware
Functions:

- Query translation and processing
- Schema integration
- Data format conversion
- Result coordination
- Transaction management across databases

MDBS Components & Execution

3. Component DBMS

Individual database management systems

Examples: Oracle, MySQL, PostgreSQL, SQL Server

Each maintains its own:

- Data models
- Query languages
- Storage formats
- Access methods

4. Local Databases

Physical data storage (shown as cylinders)

Each database contains actual data

May have different schemas and structures

Remain autonomous and independent

MDBS Components & Execution

Pros & Cons

✅ Preserves local autonomy (local DBMS independent)
✅ Enables cross-database integration

❌ Global queries are harder (need schema mappings)
❌ Mapping & query translation complex